Progressive Retrieval Attention on NVIDIA TensorRT-LLM: Semantic Memory Contracts, E0 Serving Evidence, and the Native Attachment Boundary

Jorge Simão

August 2026

Abstract

TensorRT-LLM already provides paged K/V caching, prefix reuse, in-flight batching, priority-aware retention, and external K/V connectors. Progressive Retrieval Attention (PRA) adds a different abstraction: immutable semantic resources selected by the current query rather than by prefix identity. We implement a TensorRT-LLM provider, an OpenAI-compatible selected-text path, and request-safe contracts for topology-bound native blocks, authorization, pinning, exactly-once attachment, cleanup, and prefix-pool exclusion. A live TensorRT-LLM 1.2.1 audit finds the official connector and paged-cache APIs but no public operation that attaches arbitrary selected non-prefix blocks to attention. Native E2 execution is therefore an explicit open gate, not a text fallback reported as native memory. On an RTX 5060 Laptop GPU with TinyLlama-1.1B, selected-text E0 and bounded full context both recover the controlled answer in every measured request. At eight concurrent shared requests, selected text uses 67 visible prompt tokens rather than 1,040, reaches 41.7 rather than 27.2 requests/s, and reduces TTFT p99 from 155.5 to 57.9 ms. An expanded 1,240-request sweep through concurrency 16 reaches 81.9 requests/s for independent selected inputs versus 53.6 for full context, with 50.5 versus 157.8 ms TTFT p99. A separate 360-request natural cohort covers QASPER, HotpotQA, and 2WikiMultiHopQA. These results establish the transport baseline and integration boundary needed for a later source-level E2 patch; they do not establish the economics or parity of native TensorRT-LLM PRA.

1 Introduction

Prefix caching asks whether a request begins with tokens that an engine has already processed. PRA asks which independently retained resource is relevant to the current query. The two mechanisms can cooperate, but only if three identities remain distinct:

$$I_{\text{prefix}} \neq I_{\text{PRA}} \neq I_{\text{physical block}}.$$

Conflating them creates two failure modes. A selected resource can leak into a later request through ordinary prefix reuse, or the same K/V can be included twice through both sequential and PRA paths. TensorRT-LLM is a demanding target because it already owns an optimized physical K/V system. A useful integration must add semantic identity without replacing that system.

This paper contributes three bounded results. First, it implements the shared runtime and safety contracts needed by a TensorRT-LLM adapter. Second, it audits the installed 1.2.1 runtime down to request, paged-cache-manager, and connector interfaces. Third, it measures a reproducible E0 selected-text baseline under prefix reuse and in-flight batching. The native result is a precise negative API finding: the installed public surface can load sequential request pages, but cannot expose an arbitrary set of selected non-prefix pages to attention.

2 Execution Levels and Claim Boundary

We use the shared PRA execution-level vocabulary.

E0: Selected source text is rendered into the visible prompt. Retrieval and semantic resource identity exist, but the engine performs ordinary sequential prefill.

E1: The runtime retains engine-native derived state, but model attention does not yet consume arbitrary selected non-prefix K/V.

E2: Attention consumes selected native K/V without rendering the source text into the prompt, with matched geometry and one joint normalization.

The live measurements in this paper are E0. The native classes are executable contracts and tested lifecycle machinery, not evidence that TensorRT-LLM 1.2.1 performs E2 attachment.

3 TensorRT-LLM Mapping

3.1 What the engine already owns

TensorRT-LLM owns model execution, paged K/V allocation, prefix lookup, retention, in-flight batching, and connector transfer. PRA owns logical records, routing, authorization, selected intervals, session/task policy, and HOT/WARM/COLD/SOURCE retention decisions. Physical block identifiers are derived state and are never stable resource identifiers.

3.2 Implemented adapter surface

`src/pra_tensorrt_llm/adapter.py` provides `TensorRTLLMEngineAdapter`. Without a native executor, it reports E0 and uses the supported `trtllm-serve` HTTP facade. With an explicitly installed executor it can report E2, making capability elevation a dependency injection decision rather than an inference from package availability.

`src/pra_tensorrt_llm/native.py` defines four pieces needed by a future engine seam:

1. a topology fingerprint over model revision, tensor/pipeline parallelism, K/V precision, and block size;
2. immutable logical-to-physical block handles;
3. a connector-facing store protocol with pin and unpin operations; and
4. a request attachment manager enforcing authorization, tenant isolation, exactly-once exposure, cleanup, and exclusion from ordinary prefix pools.

Tests exercise cross-tenant rejection, topology mismatch, duplicate attachment, pinned-resource removal, and cleanup after request completion.

3.3 Public API audit

The live package audit records `Request.cache_salt_id` and `kv_cache_retention_config`; paged manager operations including `get_cache_block_ids`, `pin_blocks`, and `unpin_blocks_by_id`; and the official worker/scheduler connector interfaces. These are sufficient for sequential cache reuse, retention, and external page movement. They are not sufficient for E2. Neither the request nor cache manager exposes a field or operation such as selected non-prefix block IDs or `attach_nonprefix_blocks`.

The official connector can load pages associated with request-token ranges. Treating those pages as arbitrary semantic memory would silently turn PRA into prefix restoration. A minimal E2 extension must instead carry an authorized selected-block set into request attention metadata and combine local and memory scores in one softmax. That source-level seam is future work.

4 Experimental Method

4.1 Environment

The live system uses TensorRT-LLM 1.2.1, TensorRT 10.14.1.48, PyTorch 2.9.1, and the PyTorch backend on an NVIDIA GeForce RTX 5060 Laptop GPU (compute capability 12.0, 8,151 MiB). The model is `TinyLlama/TinyLlama-1.1B-Chat-v1.0`. The server uses a 2,048-token maximum sequence, 2,048 scheduled tokens, batch size eight, 32-token K/V blocks, and a 0.30 free-memory fraction. Reported device use is 4,914 MiB during the warm E0 sweep.

4.2 Frozen conditions

The controlled resource states that the answer is `PRA_EVIDENCE_4821`. Conditions are no evidence, prefix only, selected text, prefix plus selected text, and bounded full context. The full context contains the same evidence plus six distractor resources and remains within TinyLlama's context limit. Greedy decoding emits at most 24 tokens. Each one-shot condition has five repetitions. The concurrency experiment uses five waves at concurrency 1, 2, 4, and 8 for shared and independent resources. Every logical resource is explicitly primed outside the timed window, so the concurrency table is warm steady-state. E0 and full-context conditions use the same evidence; no independent retrieval decision is made. The expanded sweep raises batch size to 16, scheduled tokens to 4,096, and the free-memory fraction to 0.50; it uses ten waves at concurrency 1, 2, 4, 8, and 16 for each context and resource-sharing condition.

4.3 Metrics and scope

We report visible and cached tokens, controlled answer success, TTFT, completion latency, requests/s, output tokens/s, and HBM snapshots. Five waves support smoke-level tails, not production percentile claims. Because E2 is blocked, native K/V bytes, connector transfer, E0/E2 parity, and disaggregated PRA transfer are intentionally absent.

5 Results

5.1 Capability gates

Table 1: Implemented capability and evidence status. Contract means the PRA-side lifecycle is executable and tested but not attached by the live TensorRT-LLM scheduler.

Capability	Status	Evidence
OpenAI selected-text E0	Ready	25 live generations
Prefix reuse and cache salt	Ready	request fields and token counters
Paged manager and connector	Available	live package introspection
PRA lifecycle/isolation	Contract ready	focused unit tests
Native selected non-prefix E2	Blocked	no public attachment hook
Disaggregated PRA E2	Not run	depends on native attachment

5.2 One-shot E0 behavior

The controls never emit the target code, while all evidence-bearing conditions do. Selected text reduces visible input by 93.6% relative to bounded full context (67 versus 1,040 tokens). Once both prompts are cached, the small smoke does not establish a TTFT advantage: warm means are 26.8 and 21.8 ms. The main benefit appears when multiple requests compete for prefill and decode capacity.

Table 2: Controlled E0 serving smoke. Visible is mean prompt tokens; times are milliseconds. Cold is the first request and warm is the mean of repetitions 2–5.

Condition	Visible	Success	Cold TTFT	Warm TTFT	Latency p50
no_prefix_no_pra	29	0.00	183.42	26.1	186.0
prefix_only	536	0.00	25.85	35.1	193.1
pra_only	67	1.00	33.98	26.8	157.8
prefix_plus_pra	574	1.00	32.33	27.1	189.2
full_context	1040	1.00	26.18	21.8	155.9

5.3 Warm in-flight batching

Table 3: Warm E0 concurrency endpoints. Every row has controlled answer success 1.00. Cached is the engine-reported mean cached-token count.

Transport / resources	C	req/s	TTFT p99	completion p99	cached
selected / shared	1	6.5	24.3	158.5	66
selected / shared	8	41.7	57.9	194.9	64
selected / independent	1	6.3	35.4	166.6	66
selected / independent	8	43.9	42.9	184.2	66
full / shared	1	6.4	26.7	159.0	1039
full / shared	8	27.2	155.5	302.8	1039
full / independent	1	6.5	22.5	156.4	1039
full / independent	8	29.0	139.7	282.6	1039

At concurrency eight with a shared resource, selected text reaches 41.7 requests/s versus 27.2 for full context, a 1.53× ratio. TTFT p99 falls from 155.5 to 57.9 ms, and completion p99 from 302.8 to 194.9 ms. The independent-resource comparison is similar: 43.9 versus 29.0 requests/s and 42.9 versus 139.7 ms TTFT p99. Engine-reported cached input is about 64–66 tokens for selected text and 1,039 for full context. Device memory remains 4,914 MiB before and after each group, so this bounded E0 test shows scheduling and token-work differences, not a measurable HBM residency reduction.

5.4 Expanded load and natural-input validation

Table 4: Expanded warm E0 endpoints. Each row contains ten measured waves; the same target record appears in selected and full context.

Workload	Context	C	Succ.	Req/s	Out tok/s	TTFT p50	TTFT p99	ITL p99
Shared	Selected	1	1.00	6.41	128.1	23.2	32.0	7.0
Shared	Selected	8	1.00	40.18	803.5	41.5	132.8	7.8
Shared	Selected	16	1.00	79.61	1592.2	46.6	58.4	7.8
Independent	Selected	1	1.00	6.53	130.5	20.4	24.8	7.0
Independent	Selected	8	1.00	40.72	814.4	50.4	60.9	8.0
Independent	Selected	16	1.00	81.92	1638.5	43.4	50.5	7.8
Shared	Full	1	1.00	6.10	121.9	27.6	35.0	7.4
Shared	Full	8	1.00	34.65	693.0	67.0	80.3	8.8
Shared	Full	16	1.00	54.77	1095.5	100.7	181.8	11.4
Independent	Full	1	1.00	6.33	126.6	22.9	25.2	7.4
Independent	Full	8	1.00	30.21	604.3	73.5	511.1	9.2
Independent	Full	16	1.00	53.60	1072.0	114.7	157.8	10.9

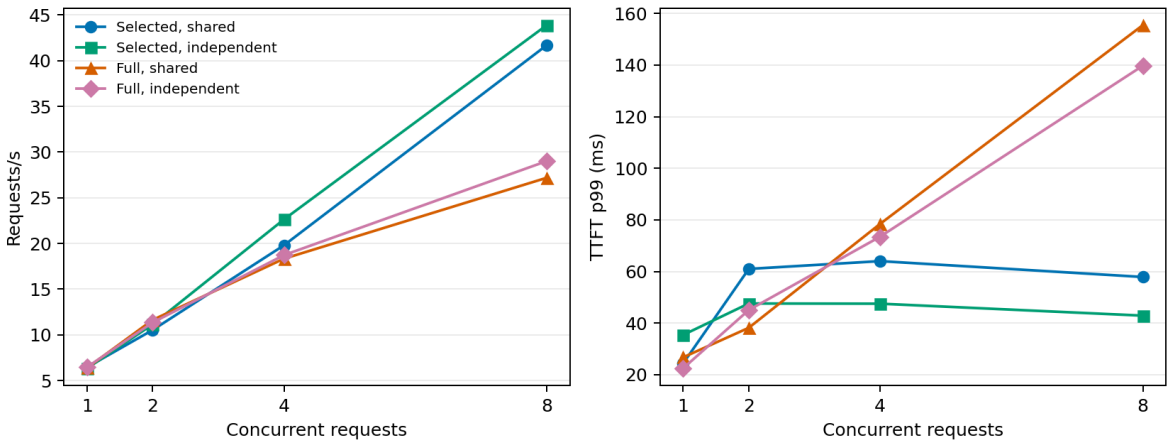

Figure 1: Warm E0 throughput and TTFT tail across in-flight load. These curves compare selected text with bounded full context; they are not native E2 results.

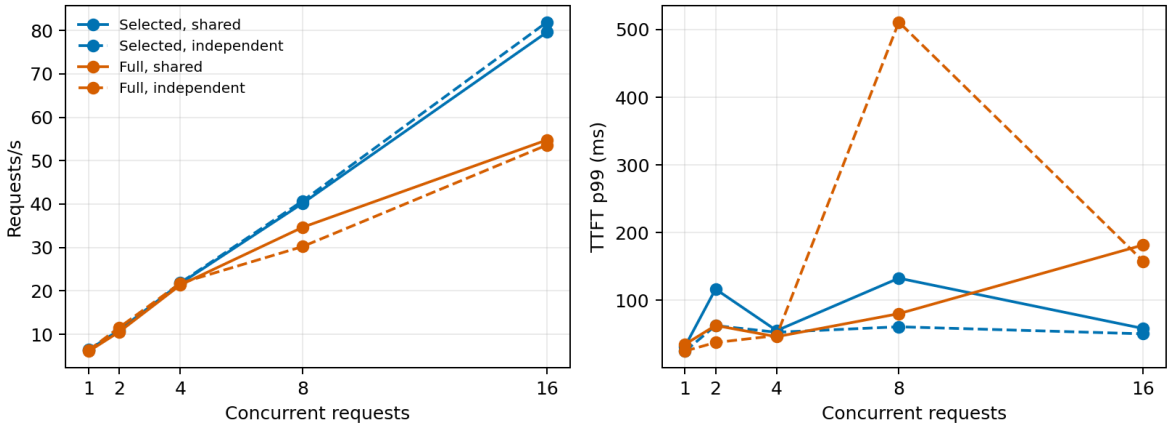

Figure 2: Expanded TensorRT-LLM E0 throughput and TTFT tails. Selected text retains its advantage through concurrency 16 for shared and independent resources.

The expanded sweep comprises 1,240 measured requests after explicit warmup. At concurrency 16, selected text reaches 79.6 requests/s with shared resources and 81.9 with independent resources. Full context reaches 54.8 and 53.6 requests/s, respectively. Selected TTFT p99 remains 50.5–58.4 ms, compared with 157.8–181.8 ms for full context. TensorRT-LLM reports 64–66 cached selected tokens and 1,039 cached full-context tokens. One independent full-context concurrency-eight group has a 511 ms TTFT p99 that does not persist at concurrency 16; we report it rather than smoothing a likely scheduler/cache disturbance out of the curve.

On 2Wiki and HotpotQA, selected evidence improves token F1 over no context and answer containment reaches 0.25 and 0.20. Full context is strongest on Hotpot (F1 0.132), while selected is slightly stronger on 2Wiki (0.105 versus 0.095). QASPER remains difficult for this checkpoint: selected and full F1 are 0.069 and 0.087. Selected median TTFT is 23.5–28.7 ms, versus 37.2–63.3 ms for full context. These are natural input and answer metrics, but TinyLlama's low absolute competence prevents a broad QA claim.

Table 5: Natural-QA E0 cohort: 20 examples per dataset and two repeats. Times are milliseconds; retrieval is frozen across execution conditions.

Dataset	Context	F1	Contain.	Prompt tok.	TTFT p50	TTFT p95
2Wiki	None	0.066	0.150	39	22.6	31.9
2Wiki	Selected	0.103	0.275	403	32.9	43.7
2Wiki	Full	0.094	0.300	940	45.6	106.3
hotpotqa	None	0.074	0.050	45	23.4	31.5
hotpotqa	Selected	0.095	0.200	438	37.7	46.2
hotpotqa	Full	0.123	0.350	1441	53.2	117.7
qasper	None	0.046	0.000	33	22.6	33.2
qasper	Selected	0.080	0.050	420	39.0	44.7
qasper	Full	0.067	0.100	1569	68.8	121.2

6 What Is Established

The experiment establishes that the shared PRA runtime can drive a TensorRT-LLM E0 endpoint without confusing semantic and prefix identity, and that smaller frozen selected-text inputs improve warm loaded serving on this hardware/model pair. It also establishes the exact point where a native integration must enter the engine. The connector is useful for storage and movement but is not itself the attention attachment mechanism.

The experiment does not establish native K/V parity, native memory savings, connector hit economics, disaggregated transfer, quantized PRA K/V, TP=2, or native-E2 QA quality. Those claims require the missing non-prefix attention seam.

7 Next Native Experiment

The next implementation should add one narrow request field containing authorized external block handles and source positions. During attention metadata construction, those blocks must remain outside prefix lookup, be pinned for the request lifetime, and participate with local K/V in one normalization. The parity ladder should then compare: ordinary full prefix, geometry-matched cached source, full native source, sparse multi-resource source, and cached decode. Only after exact/logit parity is acceptable should connector transfer, eviction hints, disaggregation, and quantized K/V be benchmarked.

8 Limitations

The controlled code task is synthetic; the additional natural cohort is small and uses a weak QA checkpoint. The original tail estimates use five waves and the expanded endpoints use ten. Only one model, one GPU, one TensorRT-LLM version, and one topology are measured. The full context is bounded to the model's 2,048-token limit. Most importantly, all generation evidence is E0.

9 Conclusion

TensorRT-LLM supplies the physical K/V machinery PRA should reuse. The current integration supplies semantic identity, safe lifecycle contracts, and a measured selected-text baseline. On the tested warm workload, smaller selected inputs improve throughput and TTFT under load. The installed public API does not yet expose arbitrary non-prefix K/V attachment, so native PRA remains a well-specified source-level integration task rather than a completed result.

Reproducibility

Raw JSON, generated tables and plots, environment audit, launch configuration, and experiment scripts are under `docs/papers/shared/results/paper6_4_tensorrt_llm` and `experiments/paper6_4_tensorrt_llm`.

References

[1] NVIDIA. *TensorRT-LLM Documentation*.
`https://nvidia.github.io/TensorRT-LLM/`

[2] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP, 2023.